

AI in images and video processing

Thomas de Jaeger
thomas.de-jaeger@epita.fr

Filtering, Edge Detection & Shape Detection

Session 2

Filtering, Edge Detection & Shape Detection

Session 2

Objectives:

- Explain how filtering affects image data and pixel values
- Implement various smoothing and sharpening filters using OpenCV
- Use gradient-based techniques to extract edges
- Understand and apply contour-based shape detection
- Detect lines and circles using Hough Transforms

What is Filtering?

Filtering is the process of modifying or enhancing an image by applying mathematical operations to pixel neighborhoods.

Main uses:

- Smoothing: Reduce noise and details
- Sharpening: Enhance contrast and highlight edges
- Edge Detection: Identify transitions in intensity



Noisy image



Filtered by Mean Filter



Filtered by Median Filter

How filtering works?

Filtering works by sliding a small matrix (kernel) across the image and computing a weighted sum.

A **kernel** (also called a filter or mask) is a small matrix used to transform images. It is applied over the image using a sliding window method.

- Example: 3x3 Box Blur Kernel:

$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$

$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$

$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} * 1/9 = \text{Averaging Filter: each output pixel becomes the average of its 8 neighbors and itself.}$

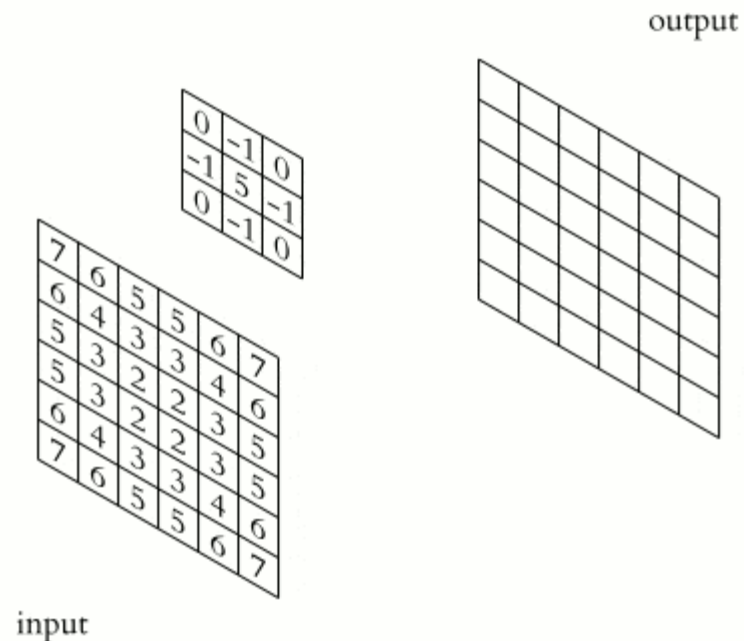
Kernels can be:

- Smoothing (e.g., average, Gaussian)
- Sharpening
- Edge detecting (e.g., Sobel, Laplacian)

Convolution?

Filtering works by sliding a small matrix (kernel) across the image and computing a weighted sum.

A **kernel** (also called a filter or mask) is a small matrix used to transform images. It is applied over the image using a sliding window method.



How to select the good kernel?

Choosing the right kernel depends on the application:

- **Noise removal:**

- Use Gaussian Blur or Median Filter (for salt-and-pepper noise)

- **Edge enhancement:**

- Use sharpening kernels (high-pass filters)

- | | |
|-----------|---|
| 0, -1, 0 | • The center pixel gets positive weight |
| -1, 5, -1 | |
| 0, -1, 0 | • Neighbors get negative weight |

- **Edge detection:**

- Use Sobel, Prewitt, or Laplacian

- **Cartoon effect or smoothing with edge preservation:**

- Use Bilateral filter

Size (3x3, 5x5...): Larger → more smoothing, less detail

You can create your own kernel!!

To design your own kernel, follow these guidelines:

Decide the purpose:	Kernel Purpose	Normalize?	Why
- Smoothing → use positive values, normalized to 1 - Sharpening → center positive, surrounding negative (high-pass) - Edge detection → sum = 0, emphasizes intensity changes	Box blur	Yes	Prevent intensity amplification
	Gaussian blur	Yes	Built-in normalization
- Kernel size: - Odd dimensions (e.g., 3x3, 5x5) so a center pixel exists	Median filter	N/A	Non-linear
	Bilateral filter	Yes	Internal normalization
- Normalize (if needed): - Ensures output pixel values stay within [0, 255] - Prevents brightening or darkening the whole image unintentionally	Sharpen	No	Keeps contrast
	Sobel, Laplacian	No	Used for gradients

Box blur filter

Purpose: Simple uniform smoothing to reduce detail and noise.

- **3x3 box kernel:**

$$K = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

- **Pixel transformation:**

$$I'(x, y) = \frac{1}{9} \sum_{i=-1}^1 \sum_{j=-1}^1 I(x + i, y + j)$$

- **OpenCV:** `cv2.blur(img, (3, 3))` # (width, height) of kernel

- ✓ Fast and easy to implement
- ✗ Blurs all edges, not ideal for preserving structure

Gaussian blur filter

Purpose: Low-pass filter used for reducing noise (high-frequency components) and for blurring regions of an image.

- **3x3 box kernel:**

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

$$\frac{1}{16}$$

1	2	1
2	4	2
1	2	1

- **Pixel transformation:**

$$I'(x, y) = \sum_{i,j} G(i, j) \cdot I(x + i, y + j)$$

- **OpenCV:** `cv2.GaussianBlur(img, (5, 5), sigmaX=1.0)`
 - (5, 5): kernel size (odd numbers)
 - sigmaX: standard deviation in X direction (Y = X if not specified)

- Low σ (e.g. 0.5): Stronger detail preservation, minimal blur
- High σ (e.g. 3.0): Stronger blur, more smoothing and loss of edges
- Large σ spreads the Gaussian curve — kernel becomes flatter (box blur)

Smoother and better edge preservation than box blur

More computational cost than simple averaging

Median blur filter

Purpose: Remove salt-and-pepper noise while preserving edges.

- No weight matrix. Instead, the kernel is treated as a window to select pixel values:

$$\Omega = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \Rightarrow I'(x, y) = \text{median}(a, b, c, d, e, f, g, h, i)$$

- **Pixel transformation:**

$$I'(x, y) = \text{median} (I(x + i, y + j) \in \Omega)$$

- **OpenCV:** `cv2.medianBlur(img, 5)`
 - 5: size of kernel (must be odd and > 1)

Excellent for impulsive noise (random white and black)
Not ideal for Gaussian noise or fine texture

Bilateral blur filter

Purpose: Smooth image while **preserving edges** using both spatial and intensity difference

- **Pixel transformation:**

$$I'(x) = \frac{1}{W_p} \sum_{i \in \Omega} I(i) \cdot f_r(|I(i) - I(x)|) \cdot f_s(\|i - x\|)$$

- the spatial kernel ensures that pixels far from the center contribute less.
- the range kernel preserves edges by ignoring very different intensity values.

- **OpenCV:** `cv2.bilateralFilter(img, d=9, sigmaColor=75, sigmaSpace=75)`

- d: Diameter of pixel neighborhood
- sigmaColor: How dissimilar colors can be to mix
- sigmaSpace: How far pixels can be to mix

Best for preserving edges and smoothing flat areas
Expensive computation time for large images

Sharpening filter

Purpose: Enhance edges and fine structures by amplifying pixel differences

It takes a smoothed image, subtracts it from the original image to obtain the "details" of the image, and adds the "details" to the original image.

- **Kernel:**
$$K = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

- **Pixel transformation:**

$$I'(x, y) = 5 \cdot I(x, y) - [I(x - 1, y) + I(x + 1, y) + I(x, y - 1) + I(x, y + 1)]$$

- **OpenCV:** `sharpened = cv2.filter2D(img, -1, kernel)`

Enhances visual sharpness and contrast

May amplify noise if image is not smoothed first

Filter summary

- **Box Blur : Uniform averaging of neighboring pixels**
 - Kernel values are all equal: $1/n^2$
 - Smooths uniformly but blurs edges
- **Gaussian Blur : Weighted average using a Gaussian distribution**
 - Weights decrease with distance from center
 - Better edge preservation than box blur
- **Median Blur: Replaces center pixel with the median of neighborhood**
 - Excellent for salt-and-pepper noise
 - Non-linear, preserves edges better
- **Bilateral Filter: Combines spatial and intensity similarity**
 - Blurs flat regions while preserving sharp edges
 - Slow but high quality
- **Sharpening Filter:** Enhance edges and fine structures by amplifying pixel differences
 - May amplify noise if image is not smoothed first
 - Enhances visual sharpness and contrast

Fast Non-Local Means Denoising

- **Purpose:** Advanced denoising that preserves textures and sharp edges by searching for **similar patches** across the image, not just in the local neighborhood

Function

`cv2.fastNlMeansDenoising()`

`cv2.fastNlMeansDenoisingColored()`

`cv2.fastNlMeansDenoisingMulti()`

`cv2.fastNlMeansDenoisingColoredMulti()`

Description

Grayscale images

RGB images

Multi-frame grayscale (e.g., video)

Multi-frame RGB

- `h`: parameter deciding filter strength. Higher `h` value removes noise better, but removes details of image also. (10 is ok)
- `templateWindowSize`: Size of patch (default: 7)
- `searchWindowSize`: Area to search for similar patches (default: 21)

Preserves texture and detail better than median or Gaussian filters.

Slower than simpler filters, parameters must be tuned.

Edge detection

Edges are sudden discontinuities in an image.

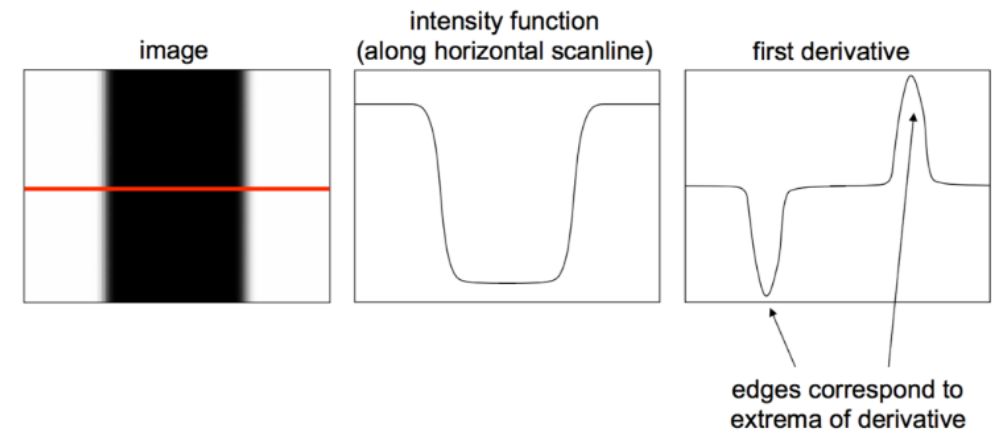
- Surface normal
- Depth
- Texture
- Color
- Illumination

Why are edges important?

- They convey semantic and shape information, useful for object recognition, geometry, and perspective.
- They are a compact representation of visual data compared to full pixel grids.

How are edges detected?

By analyzing an image's intensity profile along rows or columns. A rapid change in intensity (a strong gradient) reveals an edge — these occur where the first derivative of the intensity has local extrema.



Sobel Filter – Gradient-Based Edge Detection.

Purpose: Detect edges by computing gradient magnitude in horizontal and vertical directions. To obtain the gradients in the x (horizontal) and y (vertical) directions, apply the Sobel

Kernel:

$$G_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix} \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix}$$

Transformation: Use the gradients to compute the magnitude and direction of the edge at each pixel.

$$I'(x, y) = \sqrt{G_x^2 + G_y^2}$$

$$\theta(x, y) = \arctan\left(\frac{G_y}{G_x}\right)$$

OpenCV:

```
gx = cv2.Sobel(img_gray, cv2.CV_64F, 1, 0, ksize=3)
gy = cv2.Sobel(img_gray, cv2.CV_64F, 0, 1, ksize=3)
magnitude = cv2.magnitude(gx, gy)
```

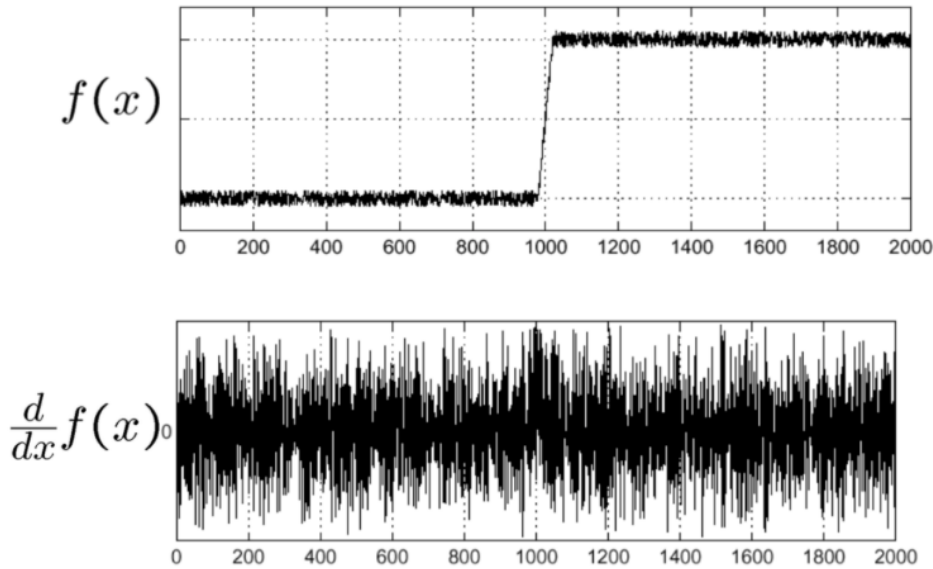
- input image (usually grayscale)
- ddepth: output data type (e.g., cv2.CV_64F for precision)
- dx: derivative order in x direction (1 to apply)
- dy: derivative order in y direction (1 to apply)
- ksize: size of extended Sobel kernel (must be odd and > 1)

Preserves texture and detail better than median or Gaussian filters.

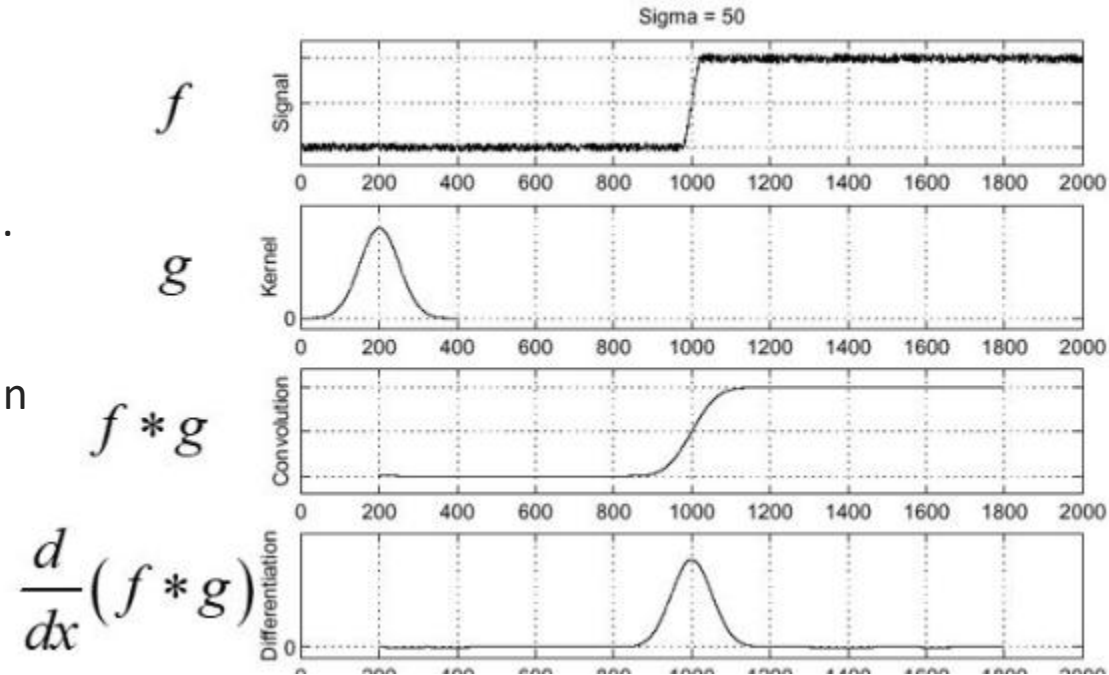
Slower than simpler filters, parameters must be tuned

Sobel Filter – Gradient-Based Edge Detection.

Real-world images are generally susceptible to a lot of noise. Consequently, plotting the pixel intensities often results in noise, making it impossible to identify where an edge is by only taking the first derivative of the function.



unable to determine the edges by the location of local maxima or minima. Need to smooth before taking its derivative. Gaussian filtering is quite popular for smoothing or blurring an image



Laplacian Filter – Second Derivative of Intensity

Purpose: Detects areas of rapid intensity change by computing the second derivative of the image. it indicates how much the function's value deviates from the average value of its surrounding points

Laplacien:

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

Kernel 4 & 8 neighbours

$$\nabla^2 I = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix} \quad \text{or} \quad \begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Transformation: $I''(x, y) = I(x + 1, y) + I(x - 1, y) + I(x, y + 1) + I(x, y - 1) - 4I(x, y)$

OpenCV: `lap = cv2.Laplacian(img_gray, cv2.CV_64F, ksize=3) # CV_64F to preserve negative gradients`
`lap_abs = cv2.convertScaleAbs(lap) #converting float to 8-bit`

Simple and fast, Isotropic (detects all edge directions).

Very sensitive to noise, Computationally intensive due to the convolution operations

Canny edge detector

Purpose: multi-stage algorithm that provides a robust solution to edge detection problems.
It includes the following steps:

- **Gaussian Blur:** Smooth the image to reduce noise using a Gaussian filter (**need to be done before**).
- **Gradient Calculation:** Compute the intensity gradients of the image using methods like Sobel
- **Non-Maximum Suppression:** Thin out the edges by suppressing all gradients that are not local maxima.
- **Double Threshold:** Identify strong and weak edges based on threshold values.
- **Edge Tracking by Hysteresis:** Finalize the edge detection by connecting weak edges to strong edges if they are part of the same edge segment

OpenCV: `canny = cv2.Canny(image, L=50, U=120)`

- L: Lower threshold value. Any value below L is considered not to be an edge.
- U: Upper threshold value. Any value above U is considered to be an edge.
 - Too high L/U → Miss fine edges (only strong ones remain)
 - Too low L/U → Include noise and unwanted details

High accuracy and robustness to noise, Good localization of edges, Produces thin edges.
Sensitive to the choice of thresholds for double thresholding

Summary of Edge Detection Methods

Why Edge Detection Matters:

- Helps identify object boundaries, textures, and structure.
- Reduces data while preserving essential shape information

Takeaways:

- Use **Sobel** for simple gradient-based edge detection in X or Y direction.
- Use **Laplacian** if direction doesn't matter but you want fast results.
- Use **Canny** for high-quality, noise-robust edge maps.

Contours detection

Contours: Curves joining all continuous points along a boundary having the same color or intensity

Why are they useful?

- Object detection
- Shape matching
- Measuring size of objects

OpenCV:

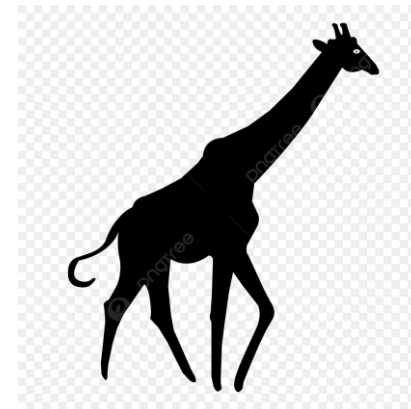
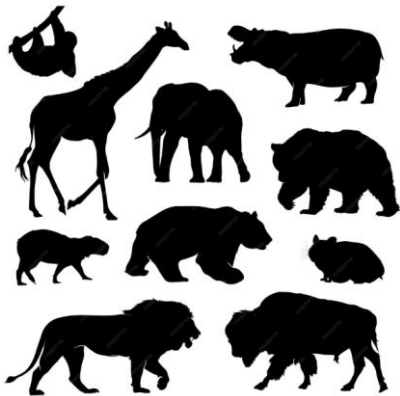
- `contours, hierarchy = cv2.findContours(thresh_img, mode, method)`
 - Mode
 - `cv2.RETR_EXTERNAL` Returns only outermost contours.
 - `cv2.RETR_LIST` Returns all contours. (no idea one is inside or outside)
 - `cv2.RETR_TREE` Returns all contours and reconstructs a full hierarchy of nested contours.
 - Method
 - `cv2.CHAIN_APPROX_NONE` Stores all contour points (most detailed).
 - `cv2.CHAIN_APPROX_SIMPLE` Compresses straight lines into endpoints only (saves memory)
- `cv2.drawContours(img, contours, -1, (0,255,0), 2) #draw the contours`

Shape matching

Purpose: Compare shapes quantitatively and determine how similar they are

OpenCV:

- `similarity = cv2.matchShapes(contour1, contour2, method, 0.0)`
 - `cv2.CONTOURS_MATCH_I1`: Sum of absolute differences
 - `cv2.CONTOURS_MATCH_I2`: Sum of squared differences
 - `cv2.CONTOURS_MATCH_I3`: Uses Hu Moments differences



Line Detection – Hough Transform

Purpose: Detect straight lines in edge-detected images by answering which points align to form a straight line

Prerequisite: Use Canny edge detection first.

Hough Transform Principle:

- A line in Cartesian space (x, y) is represented in polar coordinates (ρ, θ) .
- Each point in the image votes for all possible lines it can lie on.
- Peaks in the Hough accumulator space correspond to the most likely lines.

Standard Hough Transform is computationally expensive since it evaluates all edge points.

Probabilistic Hough Line Transform improves efficiency by only using a subset of edge points, reducing computation and allowing faster detection

OpenCV: `cv2.HoughLines()` for standard lines

`cv2.HoughLinesP()` for probabilistic version:

- `threshold` = minimum number of intersections in Hough space.
- `minLineLength` = minimum line length.
- `maxLineGap` = max gap between line segments to treat them as one line

Circle Detection – Hough Transform

Purpose: Detect circles and estimate their center and radius using the Hough Circle Transform

Prerequisite: Use Canny edge detection first.

OpenCV: `cv2.HoughCircles()`
`cv2.HoughLinesP()` for probabilistic version:

- `cv2.HOUGH_GRADIENT`: Detection method
- `dp`: Inverse ratio of accumulator resolution to image resolution (e.g., 1.2)
- `minDist`: Minimum distance between circle centers
- `param1`: Upper threshold for Canny edge detection
- `param2`: Threshold for circle detection (lower = more circles)
- `minRadius`, `maxRadius`: Optional range for radius

LAB

IA_images_Lab2_questions